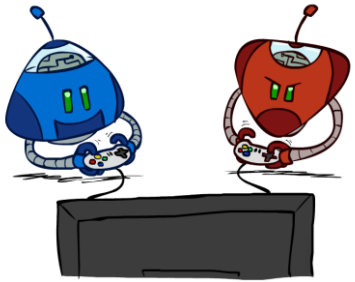
Attendance Code:
761

Game Trees: Adversarial Search

Instructor: Hyunwoo J. Kim



Adapted from CS188 of Klein and Abbeel
http://ai.berkeley.edu

Annoucement

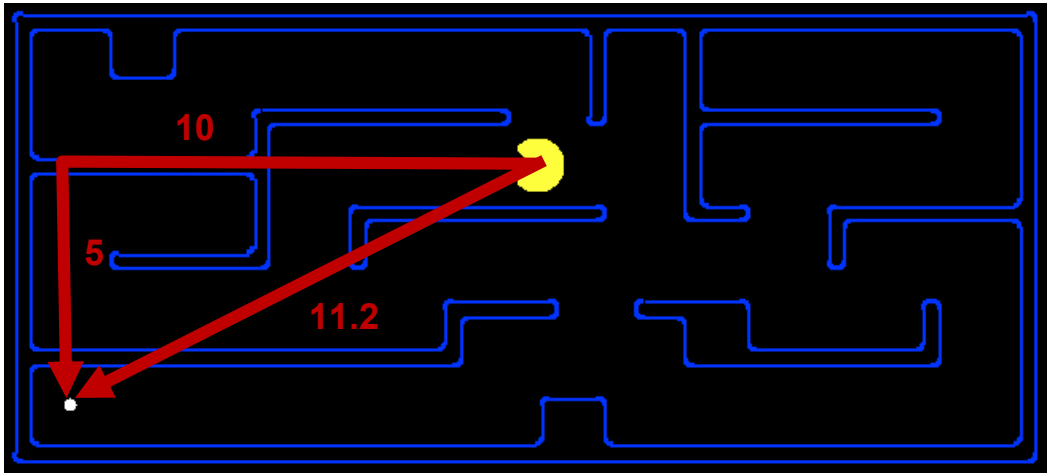
Attendance Code:

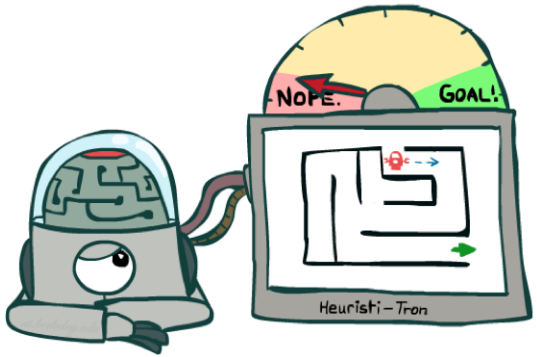
- No class this Thursday (03/26)
- Coding assignment #1 is due on 03/26

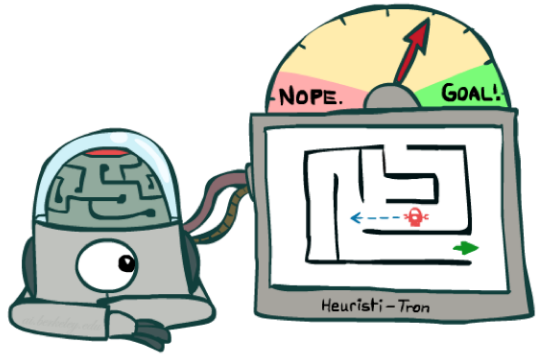
Attendance Code:
089

Search Heuristics

- A heuristic is:
  - A function that *estimates* how close a state is to a goal
  - Designed for a particular search problem
  - Examples: Manhattan distance, Euclidean distance for pathing



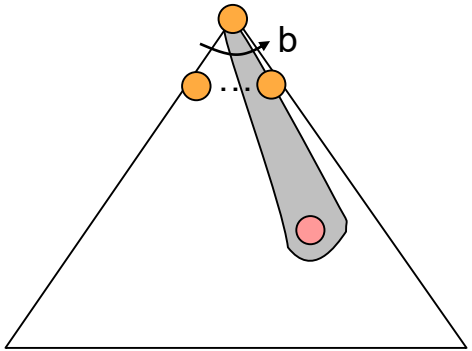




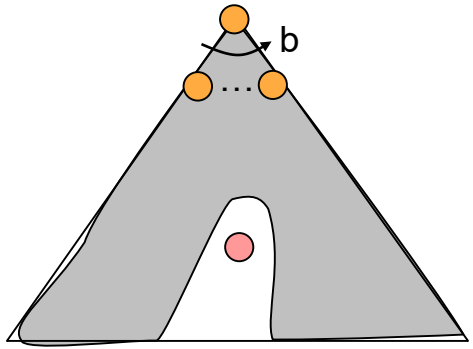
Attendance Code:
089

Greedy Search

- Strategy: expand a node that you think is closest to a goal state
  - Heuristic: estimate of distance to nearest goal for each state



- A common case:
  - Best-first takes you straight to the (wrong) goal

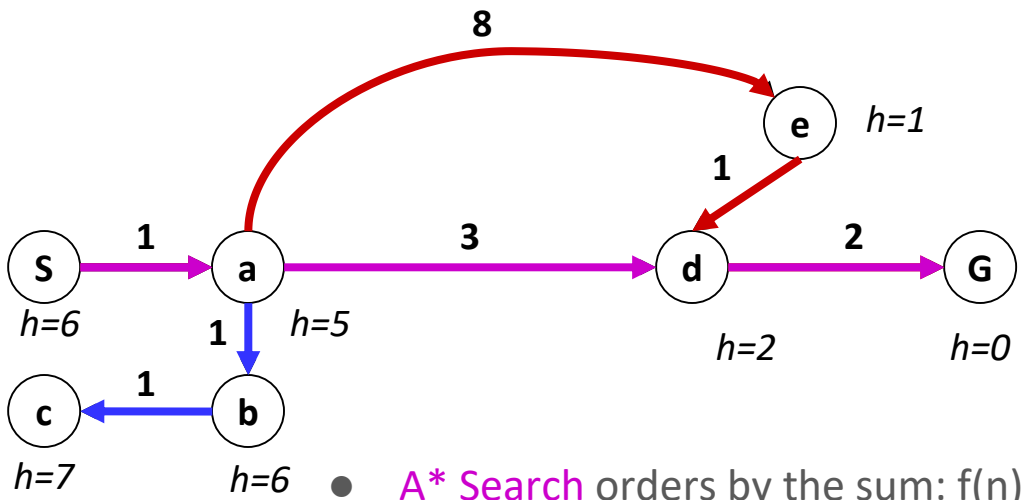


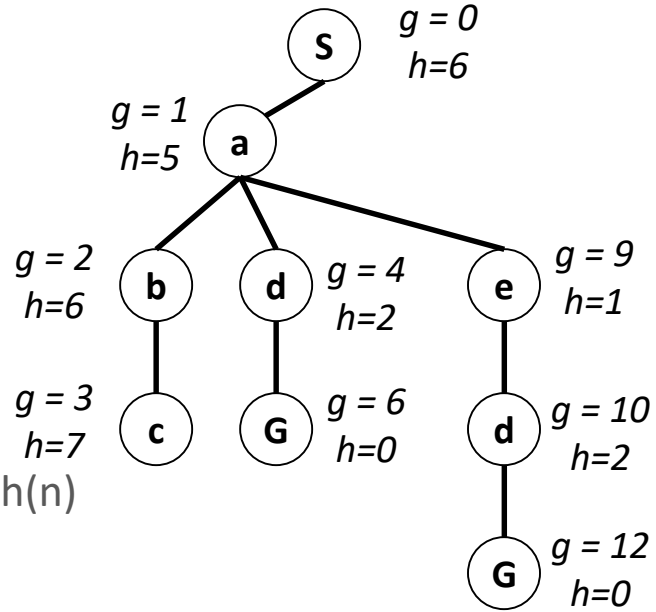
- Worst-case: like a badly-guided DFS

Attendance Code:
089

Combining UCS and Greedy

- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)



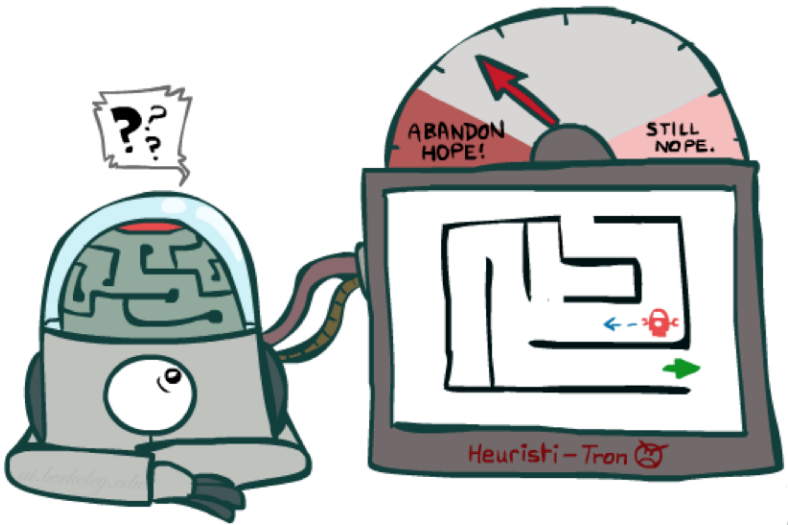


- A* Search orders by the sum: f(n) = g(n) + h(n)

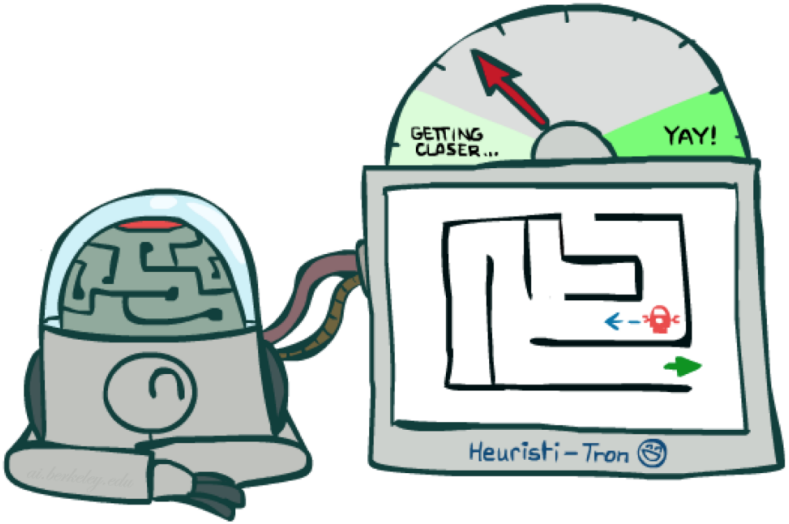
Example: Teg Grenager

Attendance Code:
089

Idea: Admissibility



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe



Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

Attendance Code:
089

Admissible Heuristics

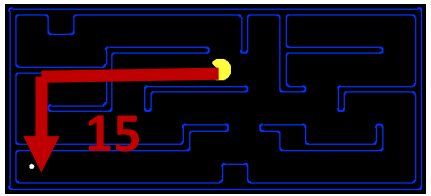
- A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal.

h=0 <= h* trival admissible heurstic. Also h=0 at goal State statisfies.

- Examples:



4

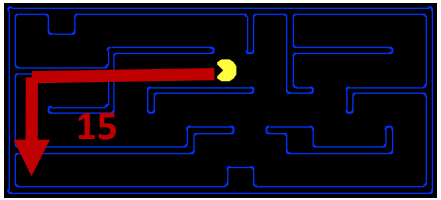
- Coming up with admissible heuristics is most of what's involved in using A* in practice.

Attendance Code:

Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics

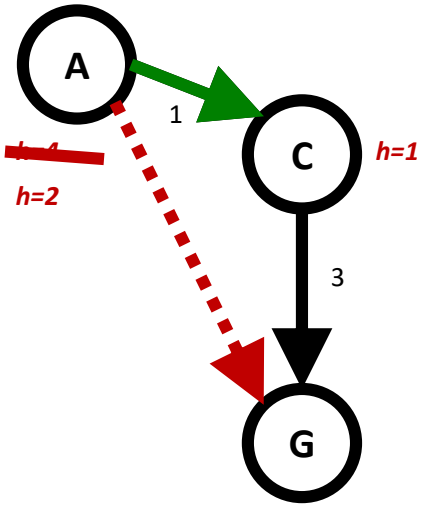
- Often, admissible heuristics are solutions to *relaxed problems,* where new actions are available (Relaxation)



- Inadmissible heuristics are often useful too

Attendance Code:

Consistency of Heuristics



- Main idea: estimated heuristic costs ≤ actual costs
  - Admissibility: heuristic cost ≤ actual cost to goal

    h(A) ≤ actual cost from A to G
  - Consistency: heuristic "arc" cost ≤ actual cost for each arc

    h(A) – h(C) ≤ cost(A to C)

- Consequences of consistency:
  - The f value along a path never decreases

    h(A) ≤ cost(A to C) + h(C)
  - A* graph search is optimal

Today's agenda

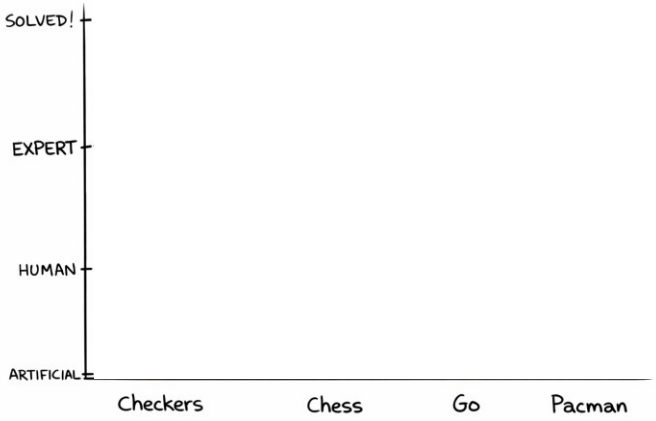
Attendance Code:
761

- Types of game:
  - Deterministic/stochastic, zero sum game, general game, cooperation
- Adversarial search (minimax)
- Resource limits
- Alpha-beta pruning

Attendance Code:
761

Game Playing State-of-the-Art

- Checkers: **1950**: First computer player. **1994**: First computer champion: Chinook ended 40-year-reign of human champion Marion Tinsley using complete 8-piece endgame. **2007: Checkers solved!**
- Chess: **1997: Deep Blue** defeats human champion Gary Kasparov in a six-game match. Deep Blue examined 200M positions per second, used very sophisticated evaluation and undisclosed methods for extending some lines of search up to 40 ply. Current programs are even better, if less historic.
- Go: Human champions are now starting to be challenged by machines, though the best humans still beat the best machines. In go, b > 300! Classic programs use pattern knowledge bases, but big recent advances use Monte Carlo (randomized) expansion methods. **2016: AlphaGo**, MCTS + human knowledge, 2017: **AlphaGo Zero,** Learning from Self-playing
- Pacman

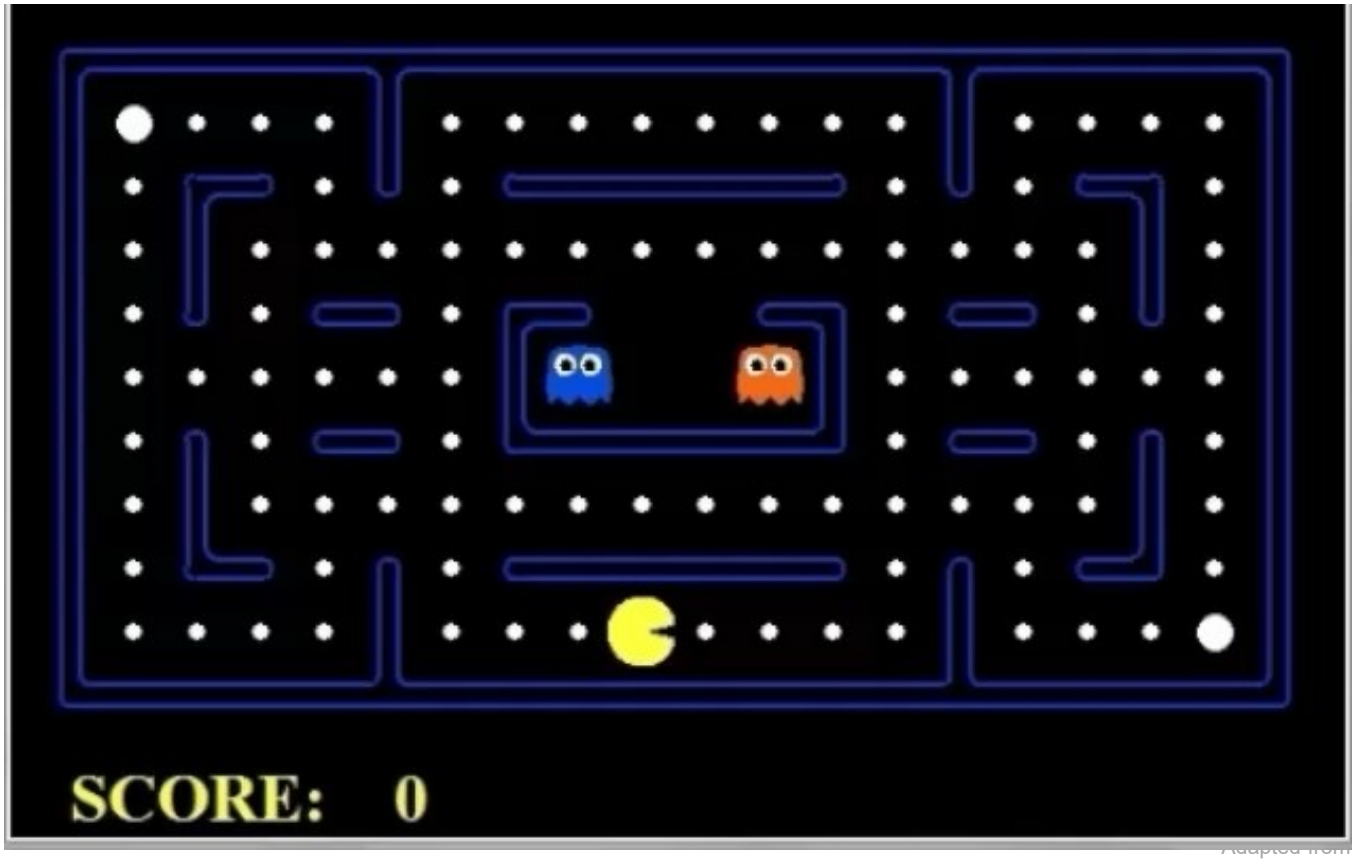


- How to play checkers https://youtu.be/ScKIdStgAfU

Behavior from **Computation**

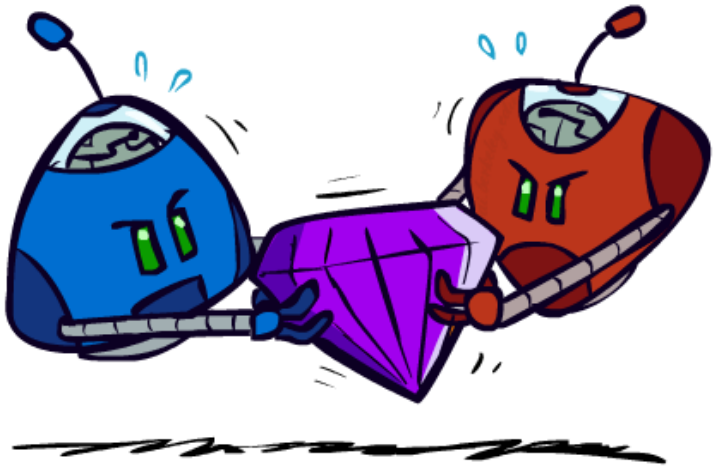
Video of Demo **Mastery** Pacman

Attendance Code:
761



Adversarial Games

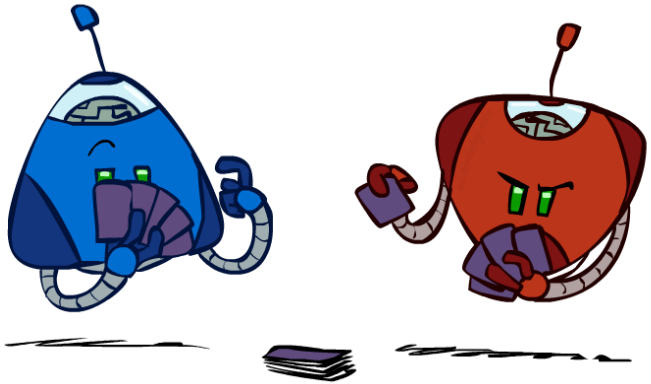
Attendance Code:
761



Attendance Code:
761

Types of Games

- Many different kinds of games!
- Axes:
  - Deterministic or stochastic?
  - One, two, or more players?
  - Zero sum?
  - Perfect information (can you see the state)?

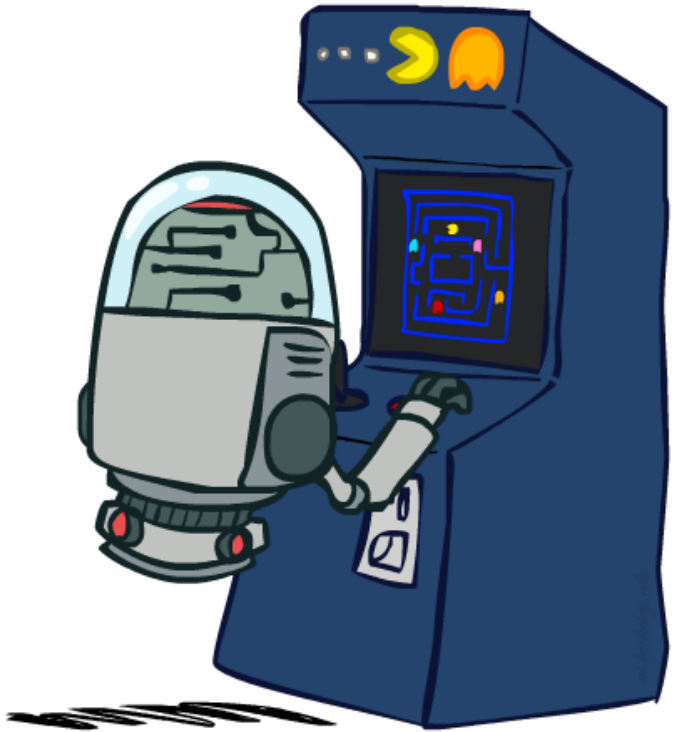


- Want algorithms for calculating a strategy (policy) which recommends a move from each state

Deterministic Games

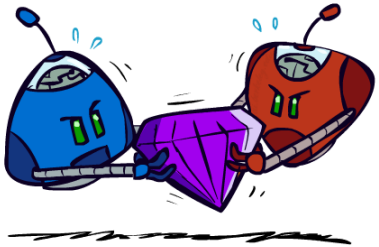
Attendance Code: 761

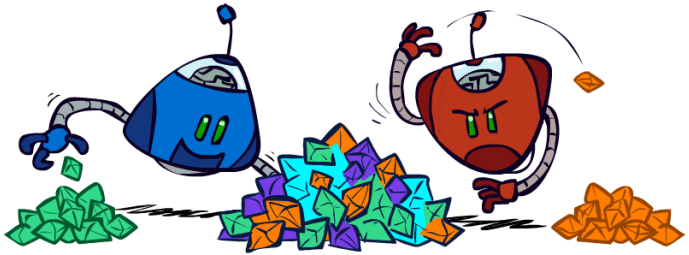
- Many possible formalizations, one is:
  - States: S (start at s_0)
  - Players: P={1...N} (usually take turns)
  - Actions: A (may depend on player / state)
  - Transition Function: SxA $\rightarrow$ S
  - Terminal Test: S $\rightarrow$ {t,f}
  - Terminal Utilities: SxP $\rightarrow$ R
- Solution for a player is a policy: S $\rightarrow$ A



Zero-Sum Games

Attendance Code:
761

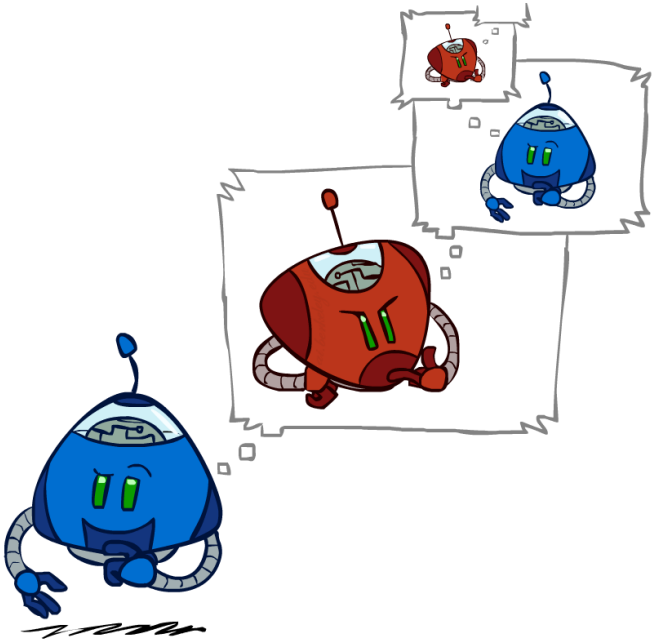




- **Zero-Sum Games**
  - Agents have opposite utilities (values on outcomes)
  - Lets us think of a single value that one maximizes and the other minimizes
  - Adversarial, pure competition
- **General Games**
  - Agents have independent utilities (values on outcomes)
  - Cooperation, indifference, competition, and more are all possible
  - More later on non-zero-sum games

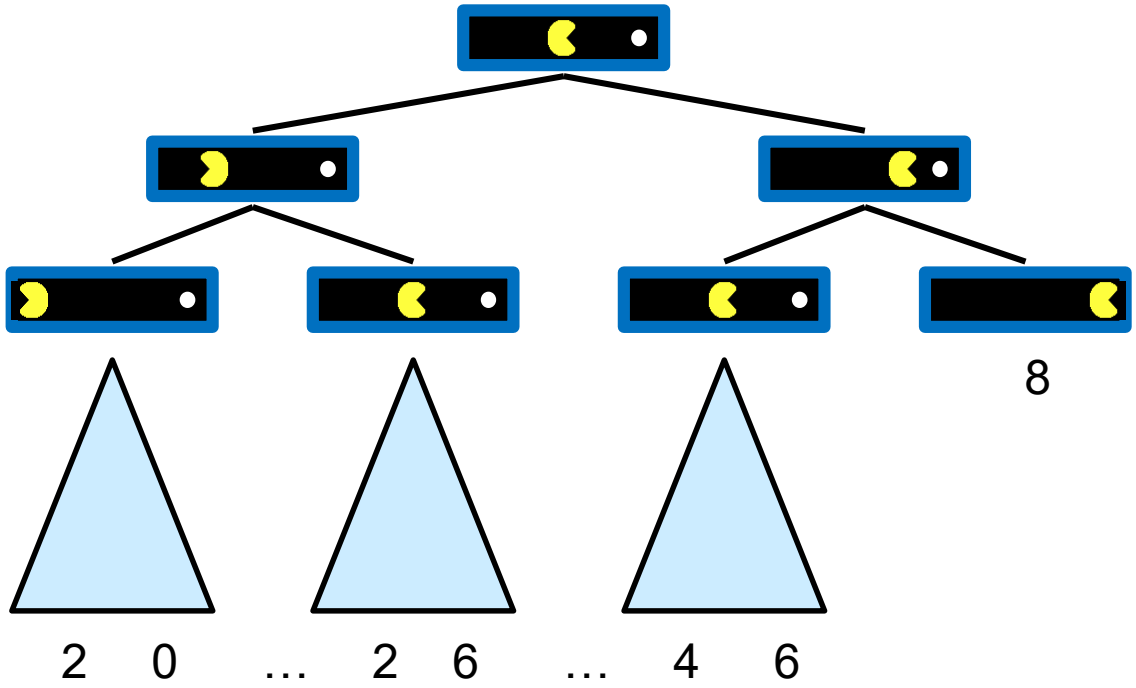
Adversarial Search

Attendance Code:
761



Single-Agent Trees

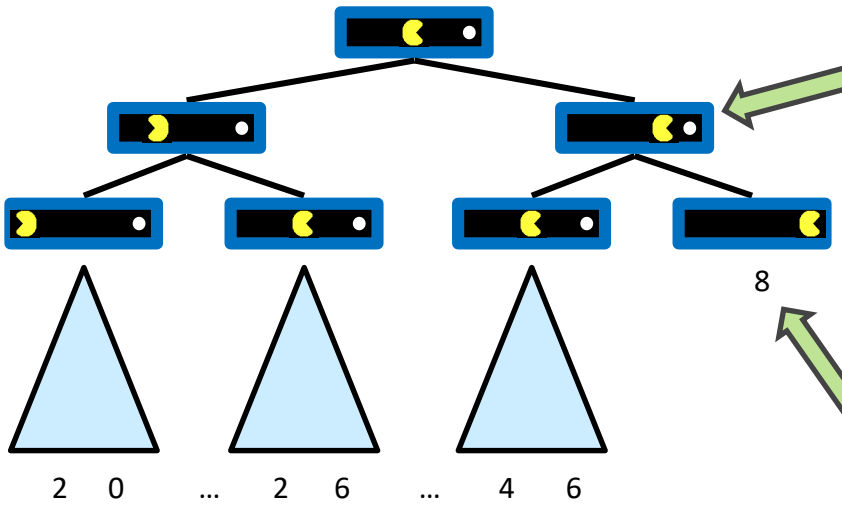
Attendance Code:
761



Value of a State

Attendance Code:
761

Value of a state:
The best achievable outcome (utility) from that state



Non-Terminal States:

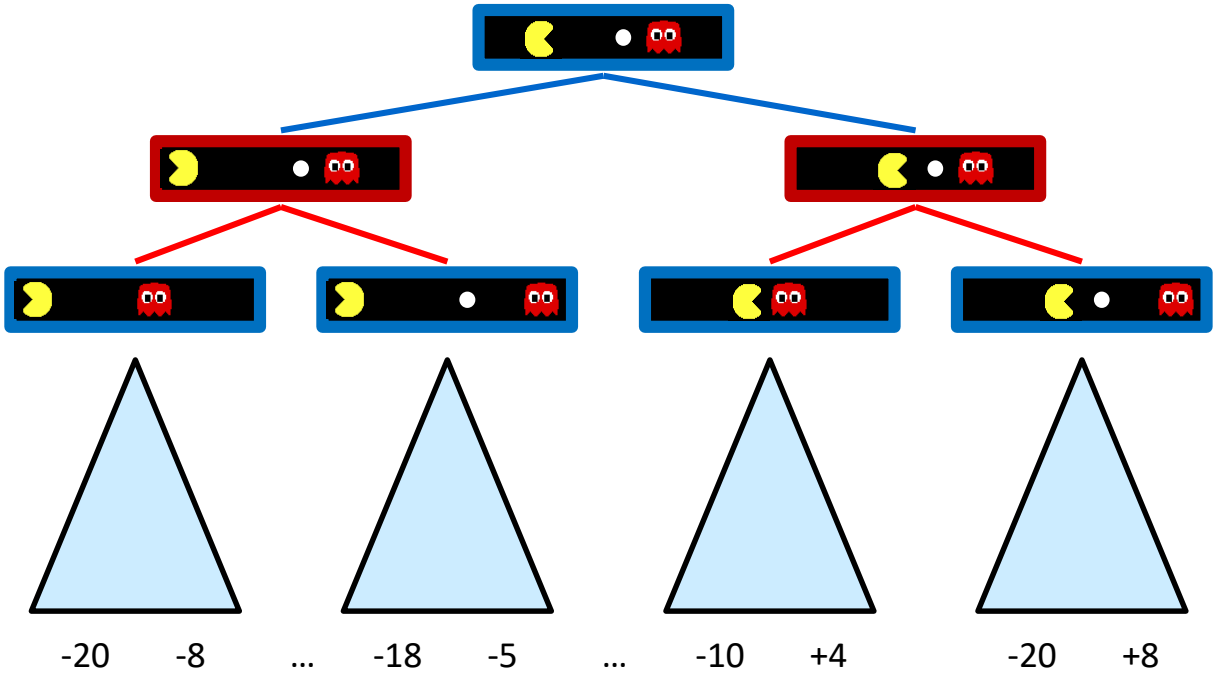
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Terminal States:

$$V(s) = \text{known}$$

Adversarial Game Trees

Attendance Code:
761



Minimax Values

Attendance Code:
761

States Under Agent's Control:

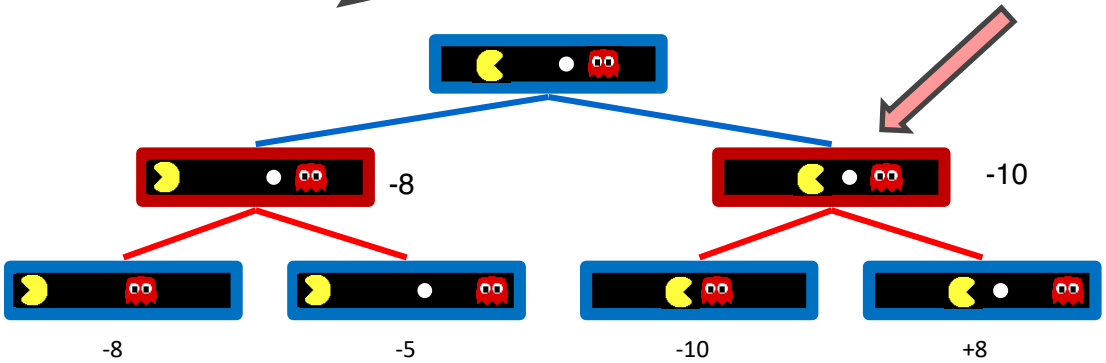
$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

pacman

ghost

States Under Opponent's Control:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

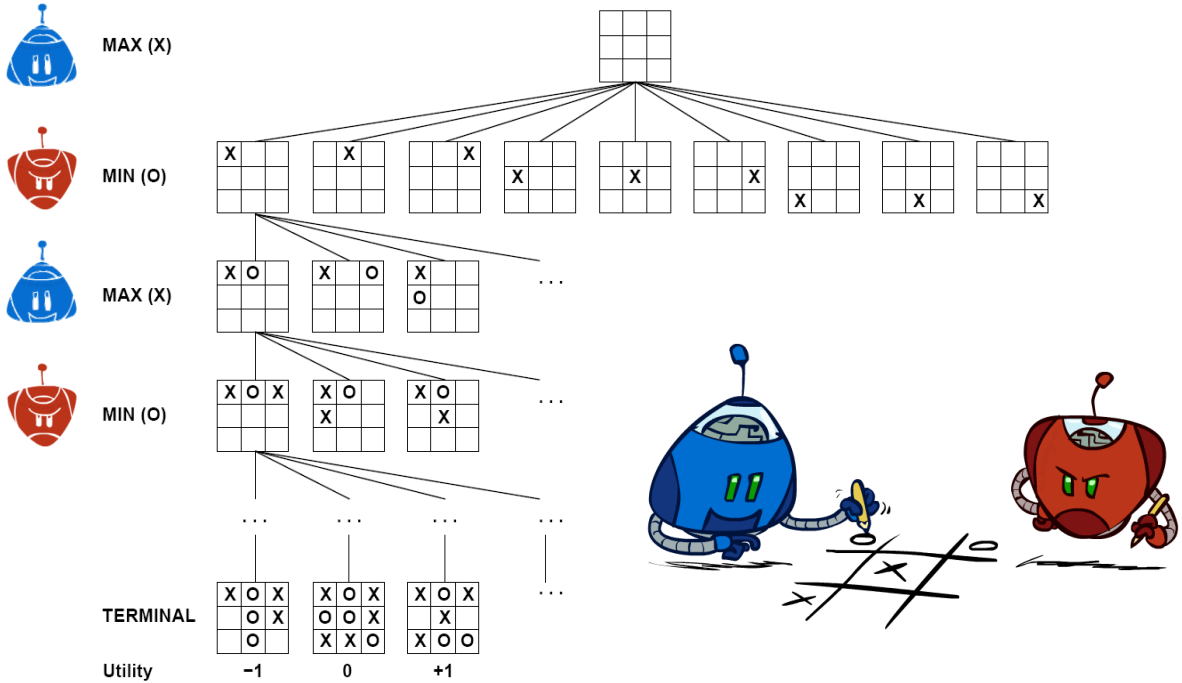


Terminal States:

$$V(s) = \text{known}$$

Tic-Tac-Toe Game Tree

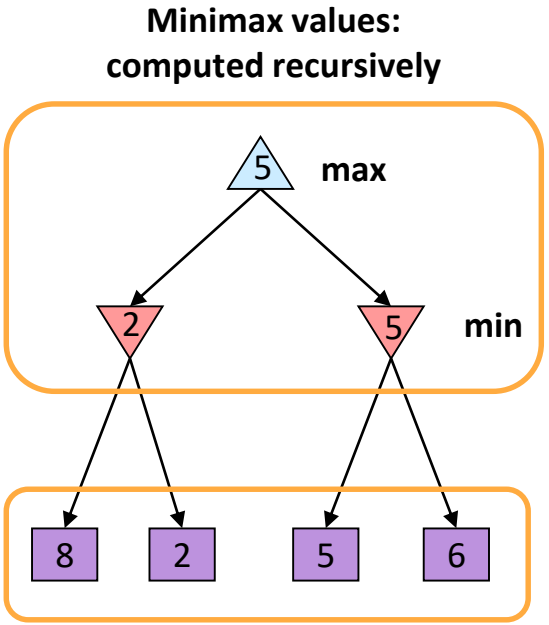
Attendance Code:
761



Adversarial Search (Minimax)

Attendance Code:
761

- Deterministic, zero-sum games:
  - Tic-tac-toe, chess, checkers
  - One player maximizes result
  - The other minimizes result
- Minimax search:
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary



Minimax Implementation

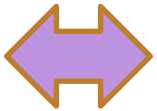
Attendance Code:
761

def max-value(state)

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```

def min-value(state)

```
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```



$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Minimax Implementation (Dispatch)

Attendance Code:
761

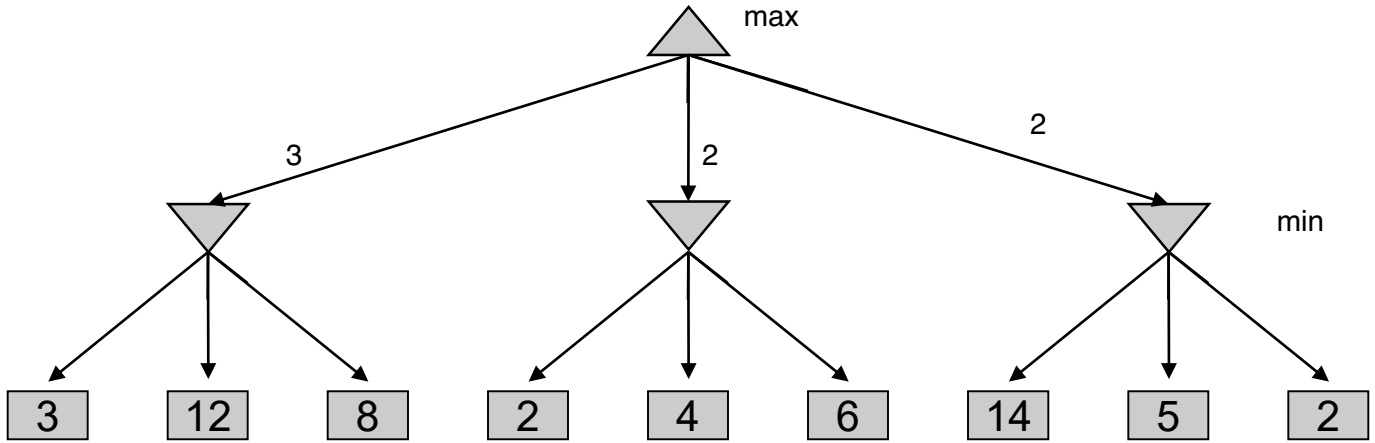
```
def value(state):
        if the state is a terminal state: return the state's utility
        if the next agent is MAX: return max-value(state)
        if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
initialize v = +∞
for each successor of state:
    v = min(v, value(successor))
return v
```

Minimax Example

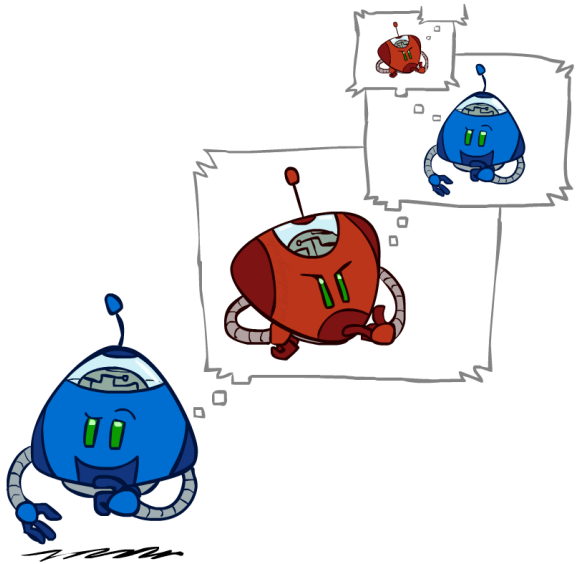
Attendance Code:
761



Minimax Efficiency

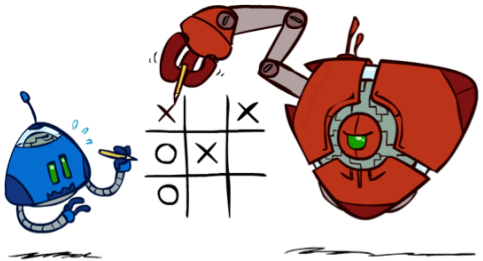
Attendance Code: 761

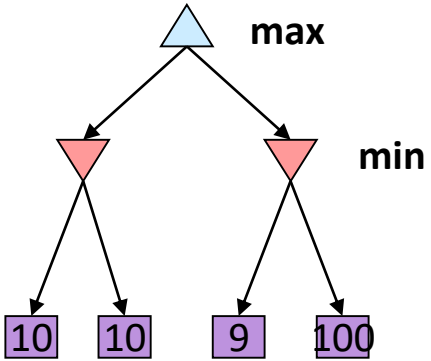
- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^m)$
  - Space: O(bm)
- Example: For chess, $b \approx 35$, $m \approx 100$
  - Exact solution is completely infeasible
  - But, do we need to explore the whole tree?

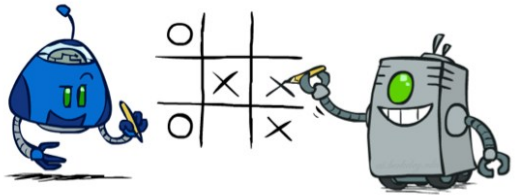


Minimax Properties

Attendance Code:
761



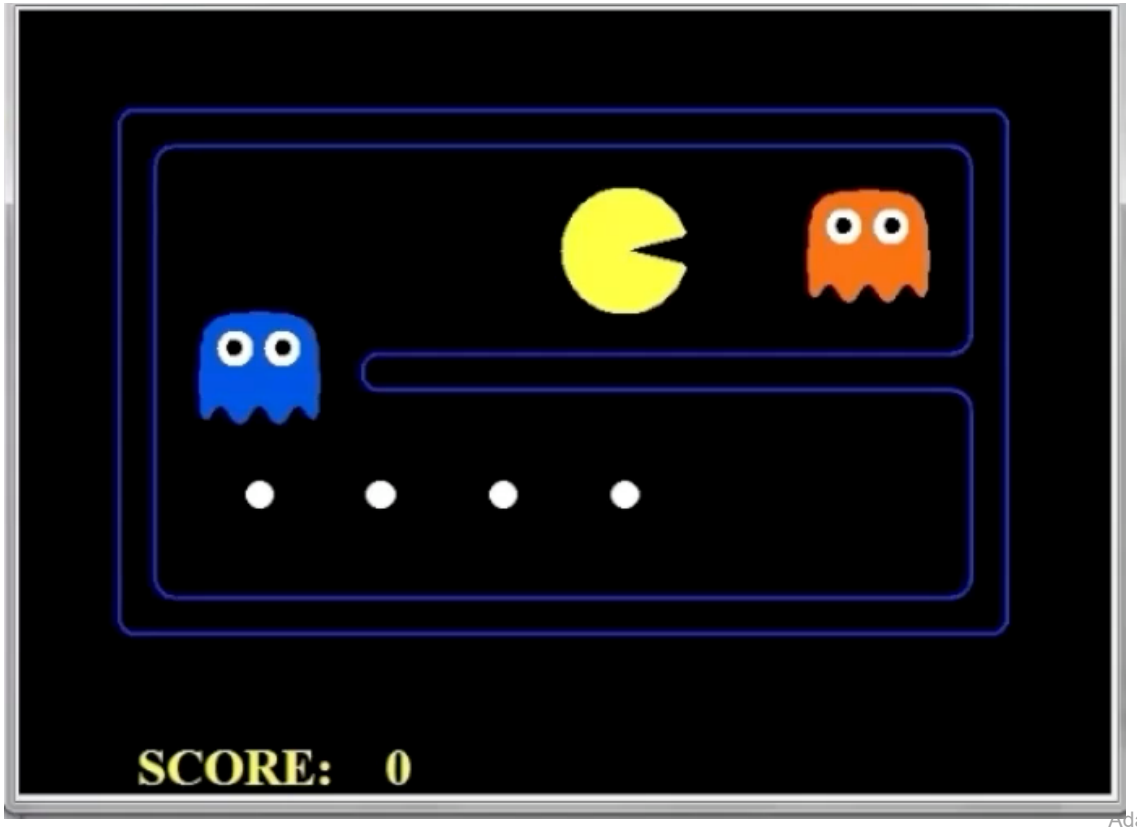




Optimal against a perfect player. Otherwise?

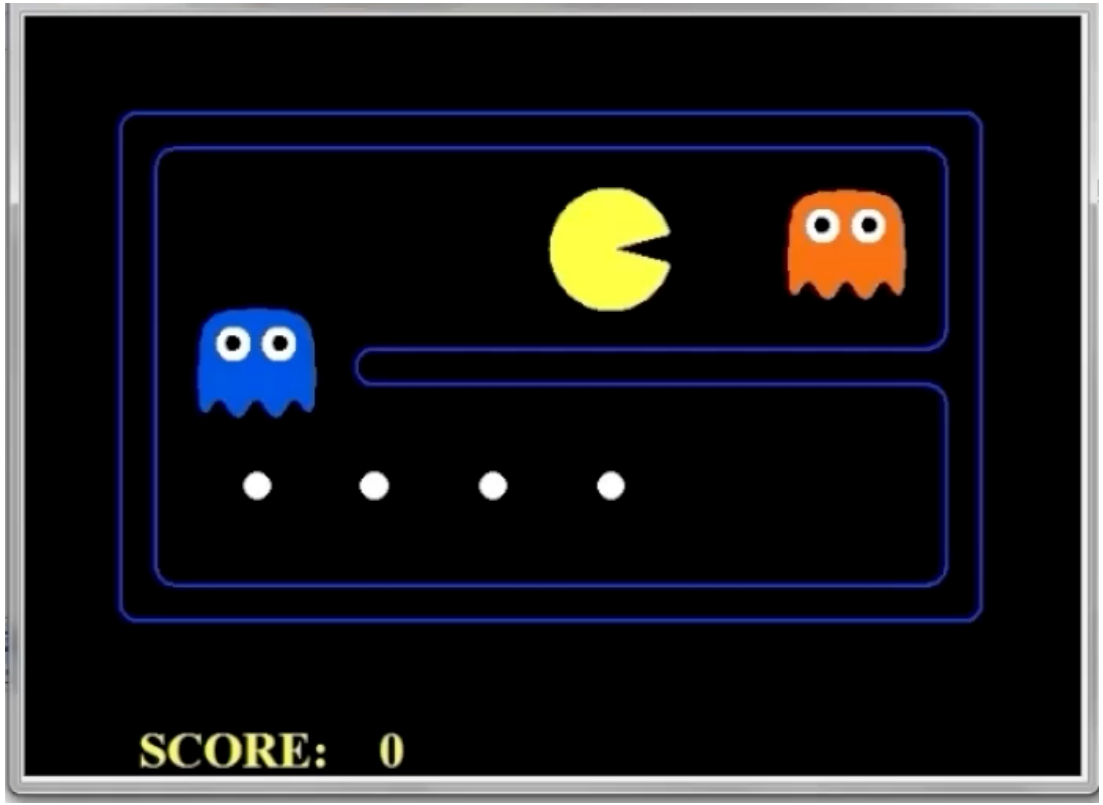
Video of Demo Min vs. Exp (Min)

Attendance Code:
761



Video of Demo Min vs. Exp (Exp)

Attendance Code:
761



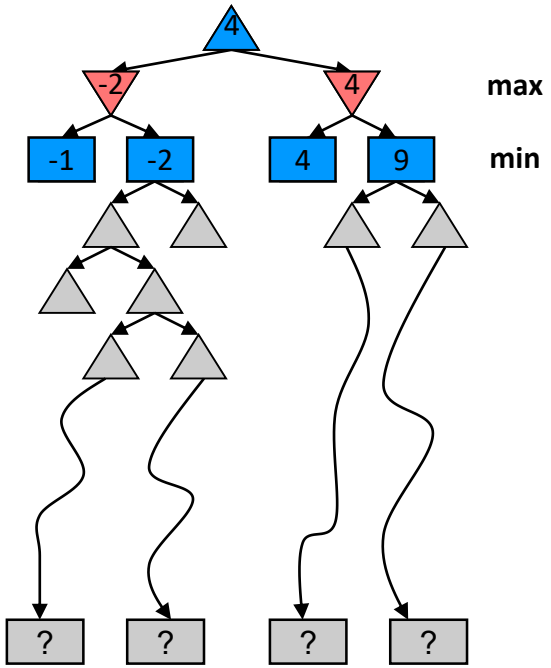
Resource Limits

Attendance Code:
761

Attendance Code:
761

Resource Limits

- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function for non-terminal positions
- Example:
  - Suppose we have 100 seconds, can explore 10K nodes / sec
  - So can check 1M nodes per move
  - α-β reaches about depth 8 – decent chess program
- Guarantee of optimal play is gone
- More plies makes a BIG difference
- Use iterative deepening for an anytime algorithm



Depth Matters

Attendance Code: 761

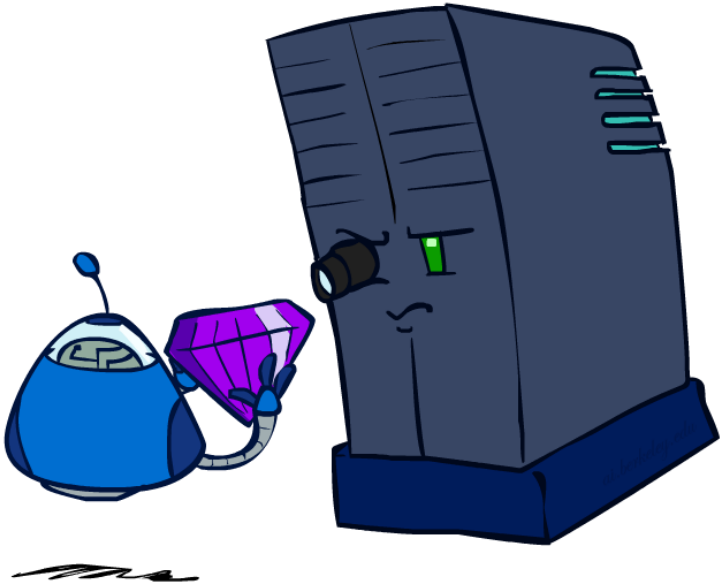
- Evaluation functions are always imperfect
- The deeper in the tree the evaluation function is buried, the less the quality of the evaluation function matters
- An important example of the tradeoff between complexity of features and complexity of computation

Attendance Code:
761

Video of Demo Limited Depth (2) vs. (10)

Evaluation Functions

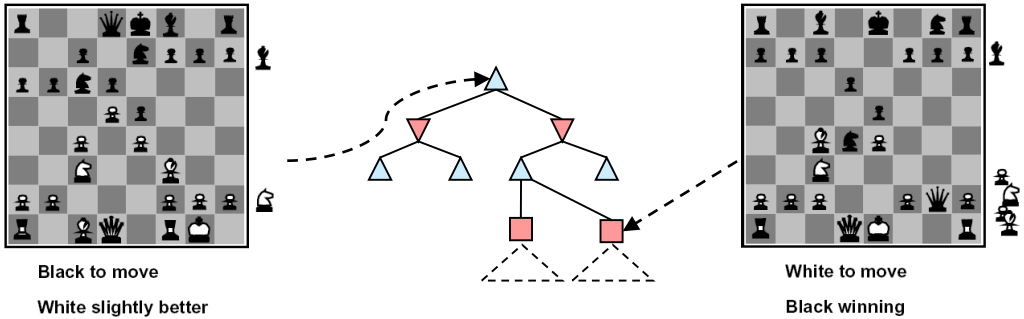
Attendance Code:
761



Evaluation Functions

Attendance Code: 761

- Evaluation functions score non-terminals in depth-limited search



- Ideal function: returns the actual minimax value of the position
- In practice: typically weighted linear sum of features:

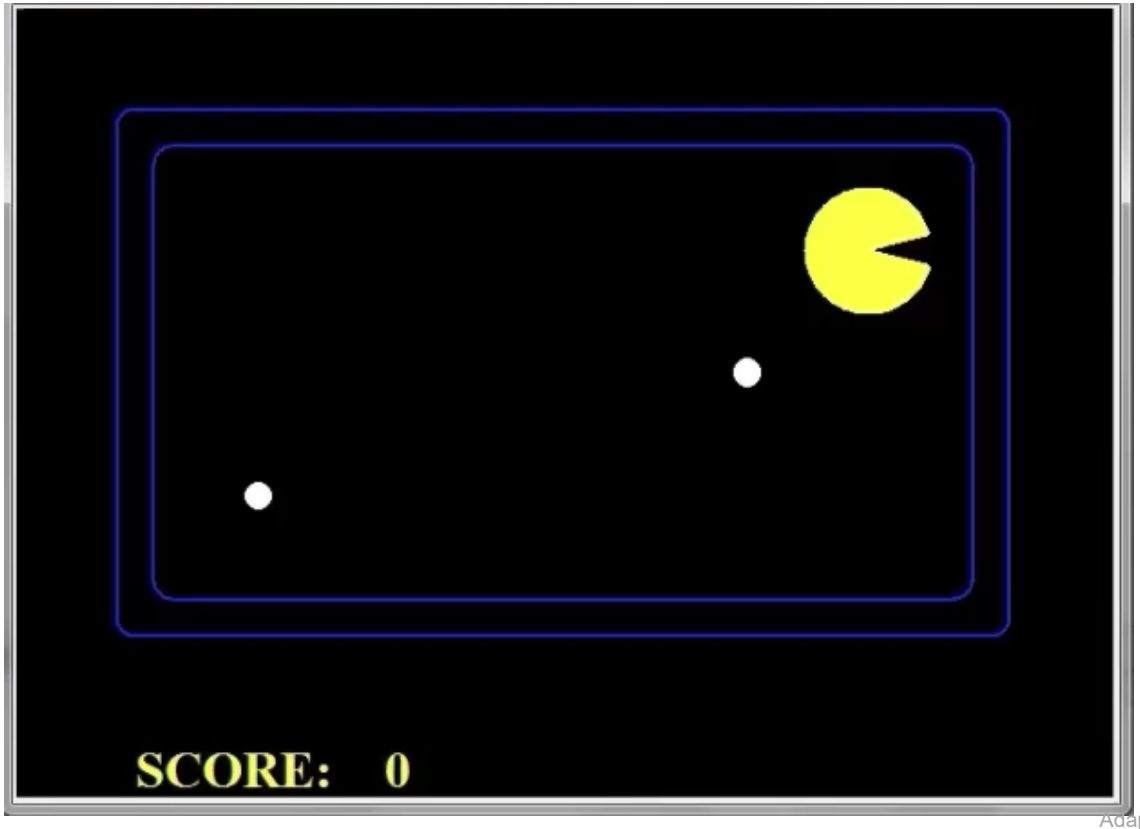
$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

- e.g. f1(s) = (num white queens – num black queens), etc.

Evaluation for Pacman

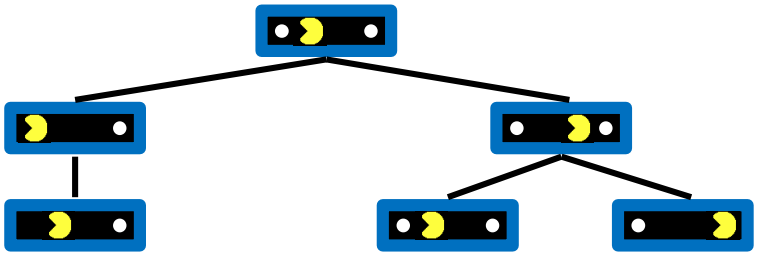
Attendance Code:
761

Video of Demo Thrashing (d=2)



Why Pacman Starves

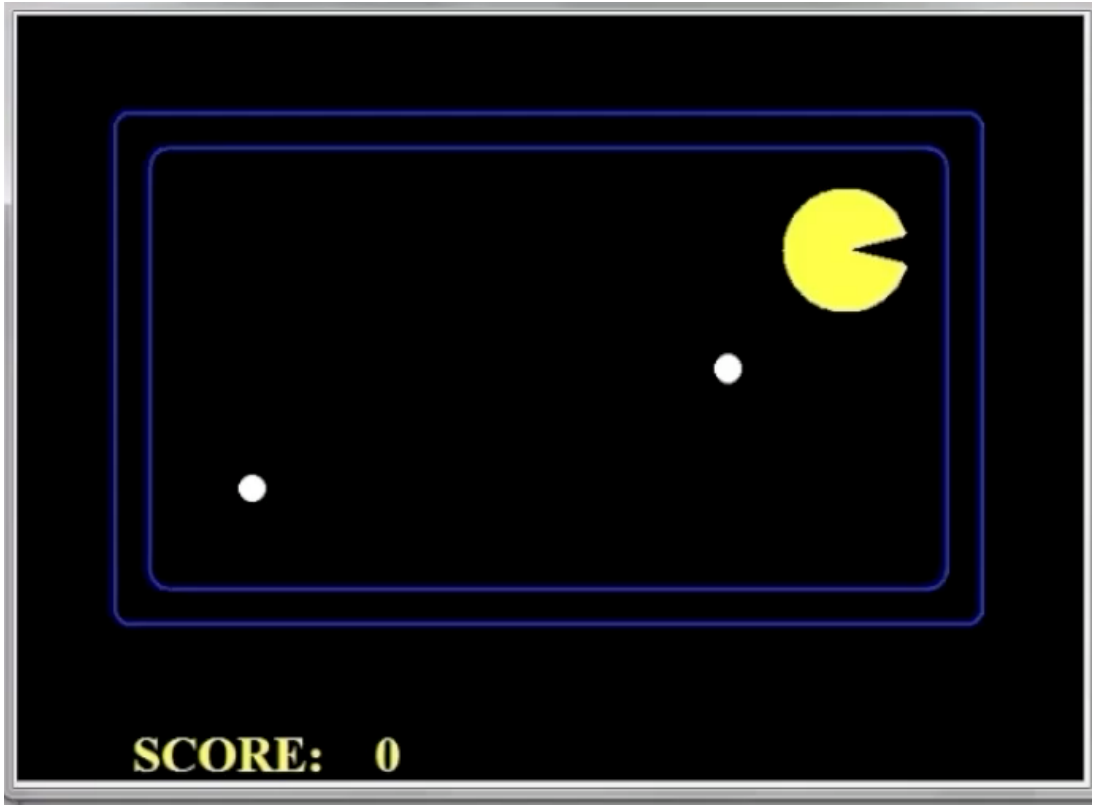
Attendance Code:
761



- A danger of replanning agents!
  - He knows his score will go up by eating the dot now (west, east)
  - He knows his score will go up just as much by eating the dot later (east, west)
  - There are no point-scoring opportunities after eating the dot (within the horizon, two here)
  - Therefore, waiting seems just as good as eating: he may go east, then back west in the next round of replanning!

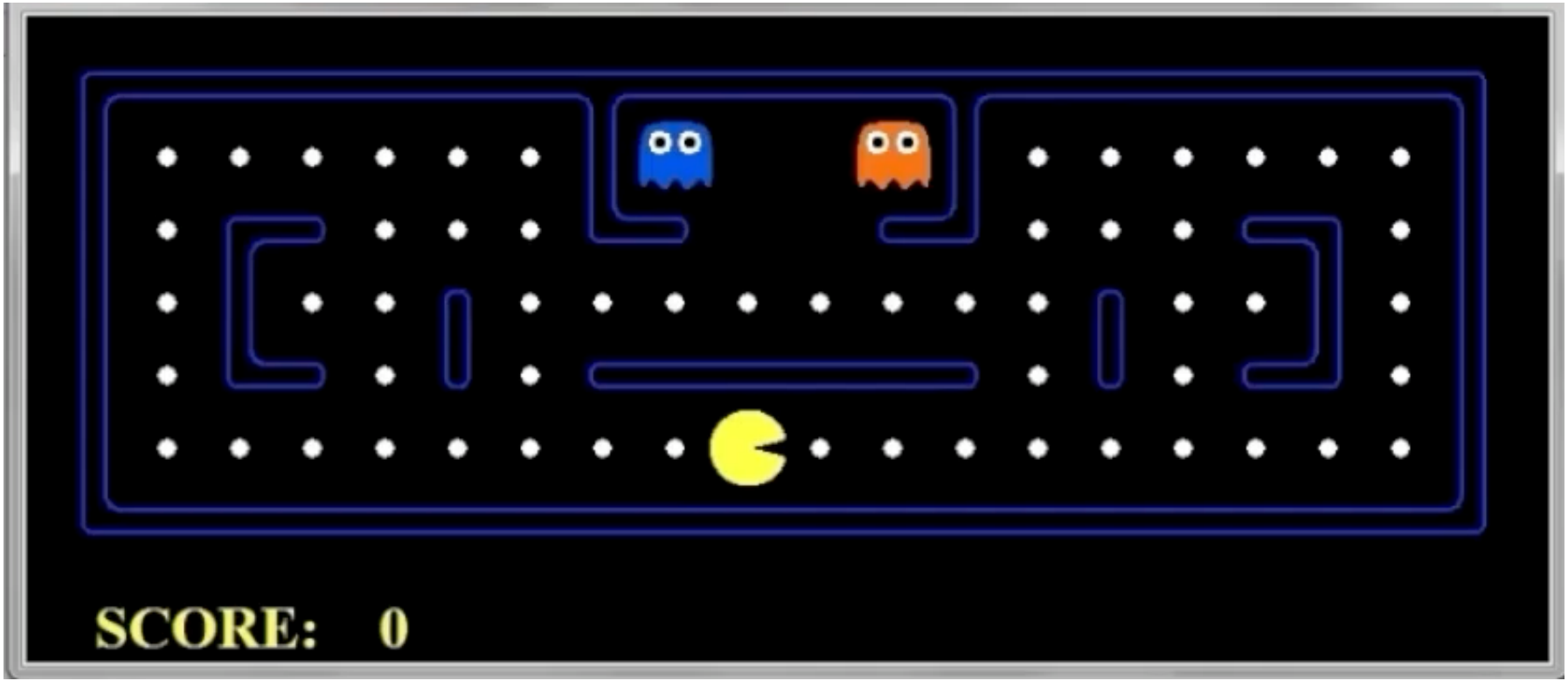
Video of Demo Thrashing -- Fixed (d=2)

Attendance Code:
761



Video of Demo Smart Ghosts (Coordination)

Attendance Code:
761

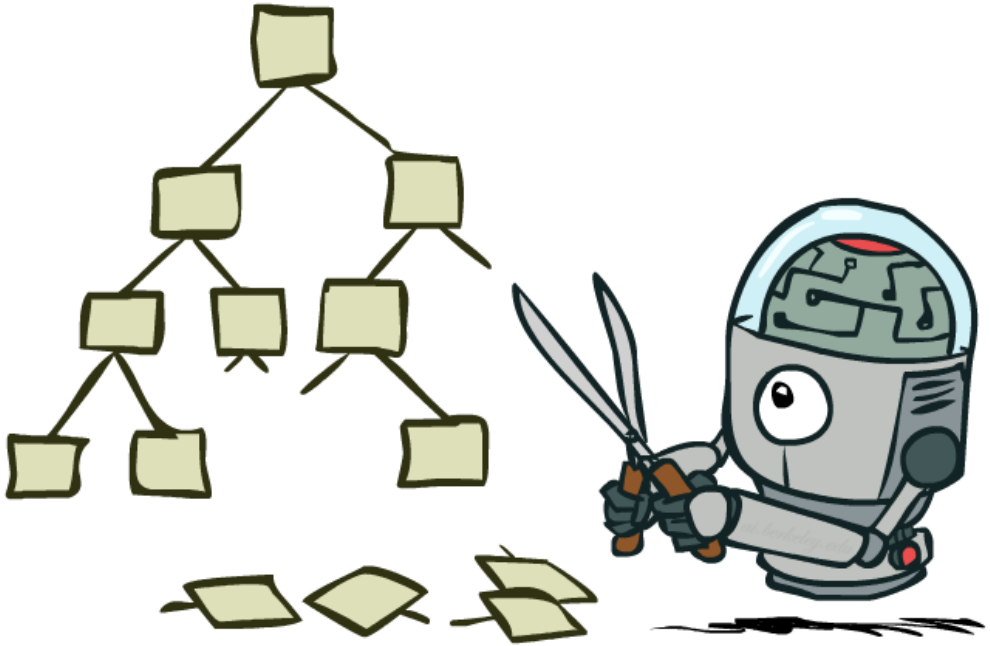


Video of Demo Smart Ghosts (Coordination) – Zoomed In

Attendance Code:
761

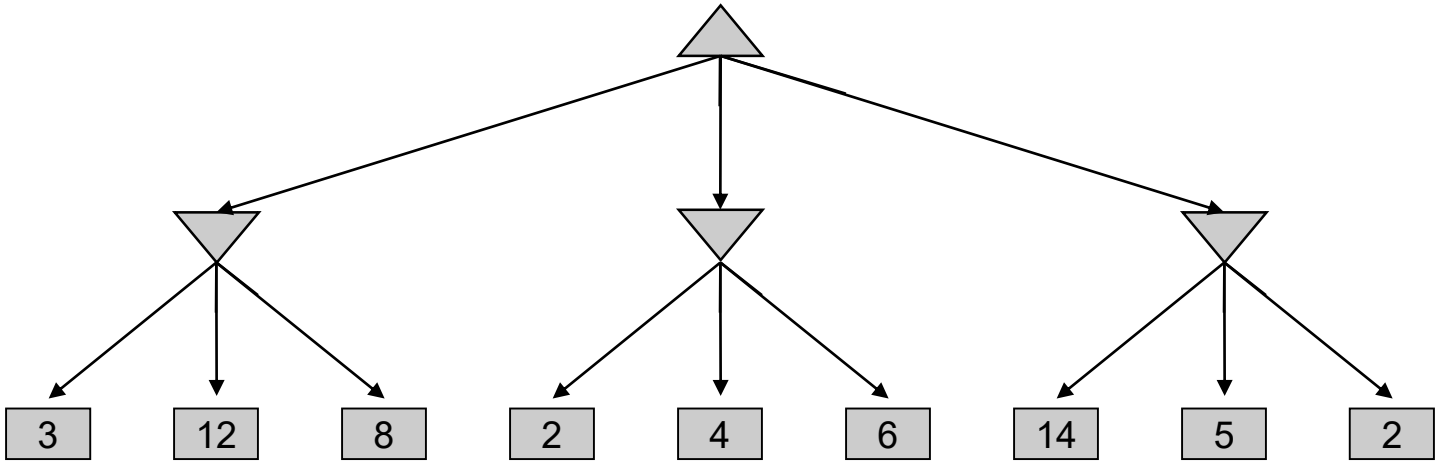
Game Tree Pruning

Attendance Code:
761



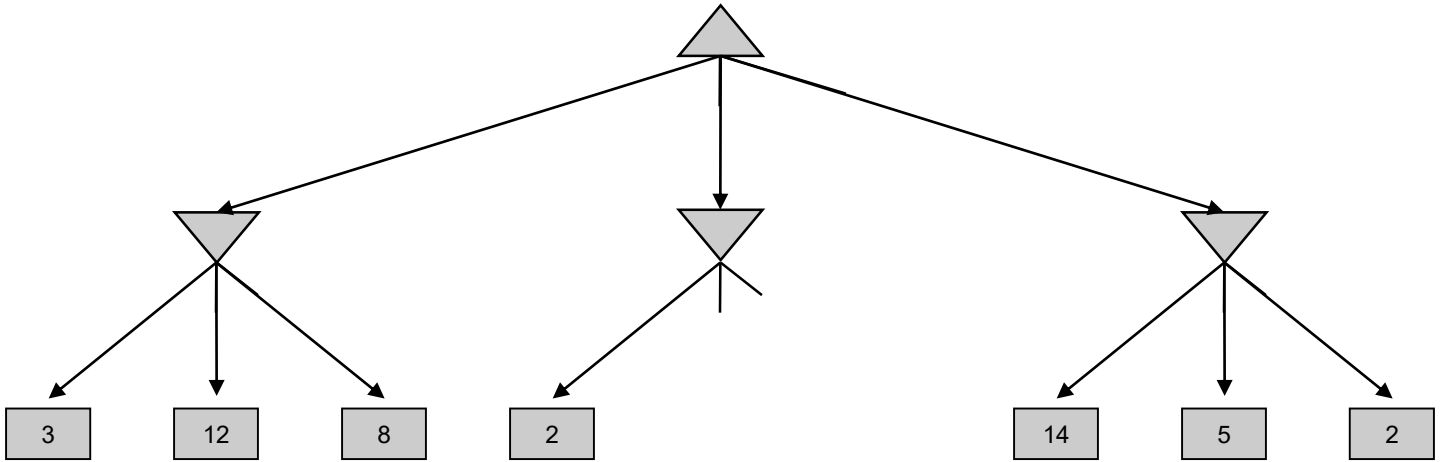
Minimax Example

Attendance Code:
761



Minimax Pruning

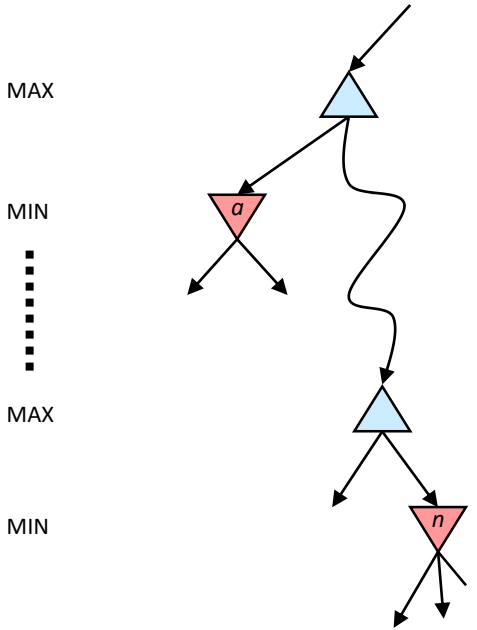
Attendance Code:
761



Alpha-Beta Pruning

Attendance Code: 761

- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node *n*
  - We're looping over *n*'s children
  - *n*'s estimate of the childrens' min is dropping
  - Who cares about n's value? MAX
  - Let *a* be the best value that MAX can get at any choice point along the current path from the root
  - If *n* becomes worse than *a*, MAX will avoid it, so we can stop considering *n*'s other children (it's already bad enough that it won't be played)
- MAX version is symmetric



Alpha-Beta Implementation

Attendance Code:
761

α: MAX's best option on path to root
β: MIN's best option on path to root

def max-value(state, \alpha, \beta)

```
initialize v = -∞
for each successor of state:
    v = max(v, value(successor, α, β))
    if v ≥ β return v
    α = max(α, v)
return v
```

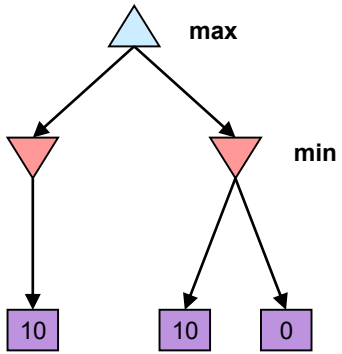
def min-value(state, \alpha, \beta)

```
initialize v = +∞
for each successor of state:
    v = min(v, value(successor, α, β))
    if v ≤ α return v
    β = min(β, v)
return v
```

Attendance Code:
761

Alpha-Beta Pruning Properties

- This pruning has no effect on minimax value computed for the root!
- Values of intermediate nodes might be wrong
  - Important: children of the root may have the wrong value
  - So the most naïve version won't let you do action selection
- Good child ordering improves effectiveness of pruning
- With "perfect ordering":
  - Time complexity drops to $O(b^{m/2})$
  - Doubles solvable depth!
- Full search of, e.g. chess, is still hopeless…
- This is a simple example of metareasoning (computing about what to compute)



Alpha-Beta Quiz

Attendance Code:
761

- Where can we prune?

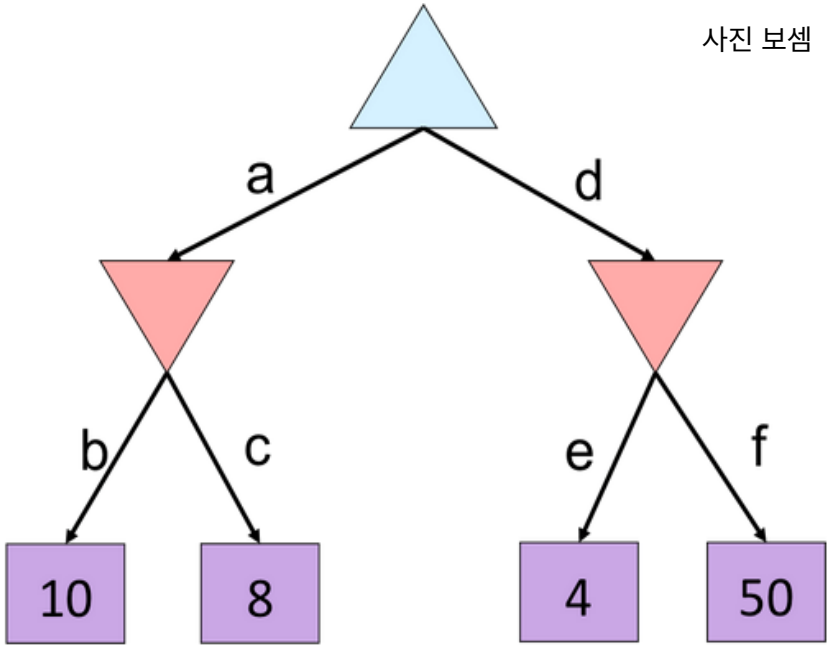
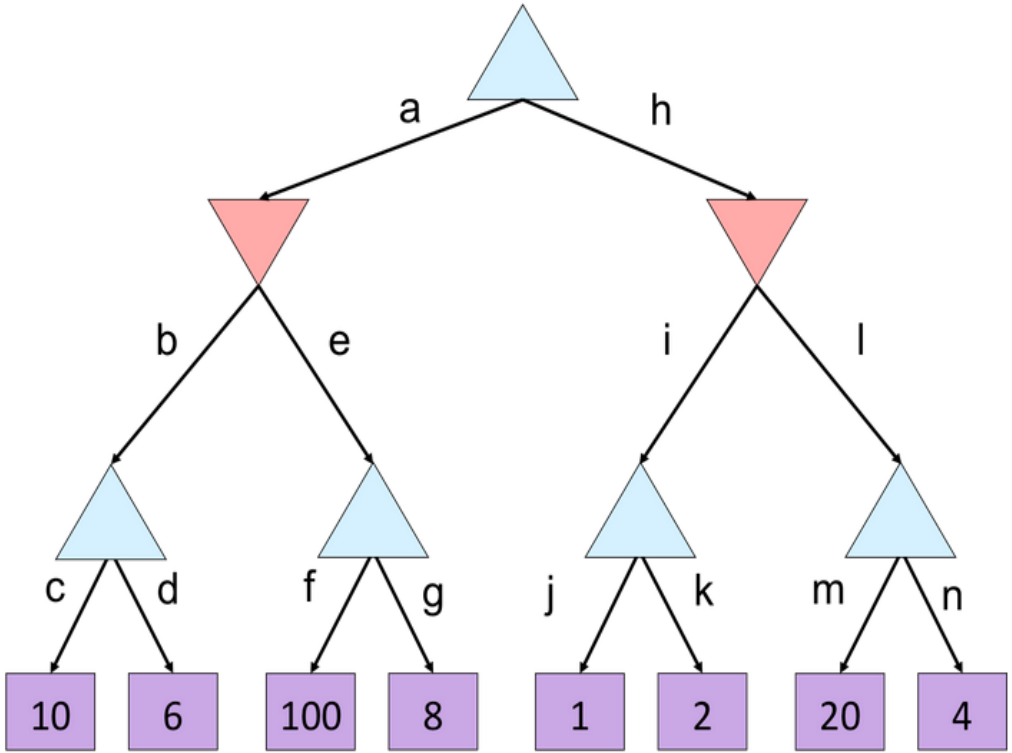


사진 보셈

Alpha-Beta Quiz 2

Attendance Code:
761



Questions

Attendance Code:
761

- What's a Game Tree?
- Can we learn evaluation functions? How about a nonlinear eval func (e.g., DL)?
- Does Alpha-beta pruning change the decision?
- Does cooperation emerge from Min-max trees?
- How can we extend the alpha-beta pruning to multi-agent game?

What's next?

Attendance Code:
761

- Uncertainty!